

AI Assisted Coding

B.GEETHANJALI

(2303A51005)

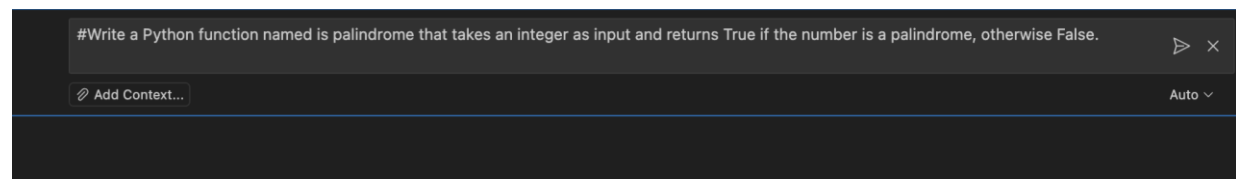
BATCH-01

Lab Experiment: Prompt Engineering – Improving Prompts and Context Management

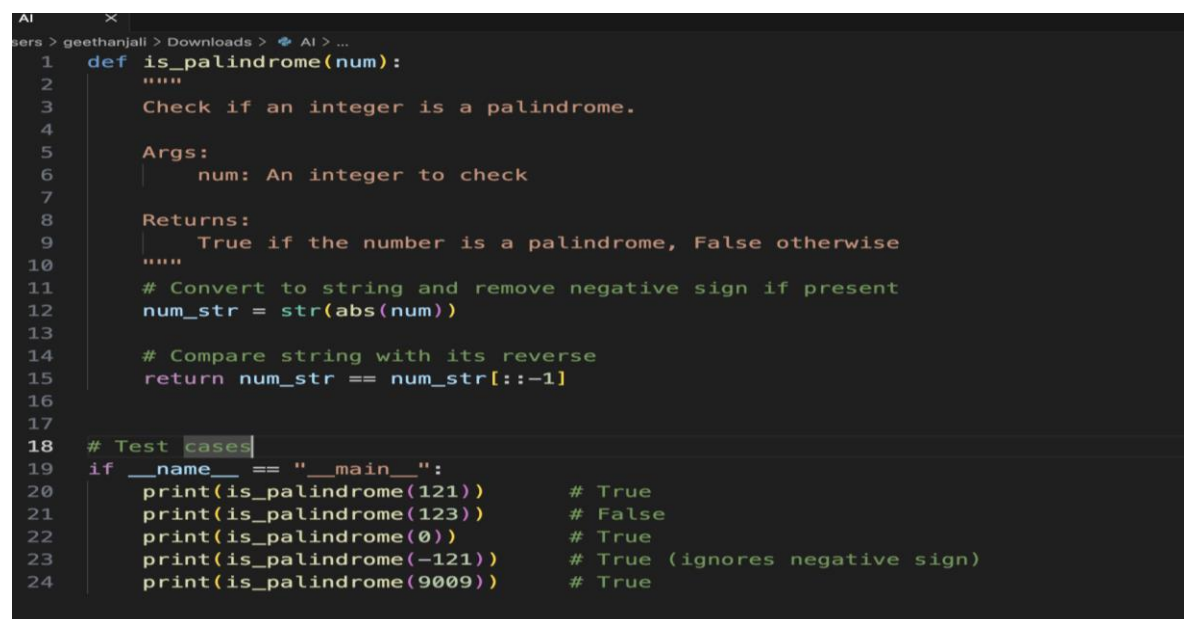
Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Prompt:



Copilot's suggestion:



Output:

```
geethanjali@Geethanjali / % print(is_palindrome(121))
zsh: no matches found: print(is_palindrome(121))
geethanjali@Geethanjali / % print(is_palindrome(121))

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % clear
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
True
False
True
True
True
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
True
False
True
True
True
geethanjali@Geethanjali / %
```

Logical errors or missing edge-case handling: Correctly identifies palindromes for: Positive numbers , Single-digit numbers , Zero ,Logic for reversing the number is correct.

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Prompt:

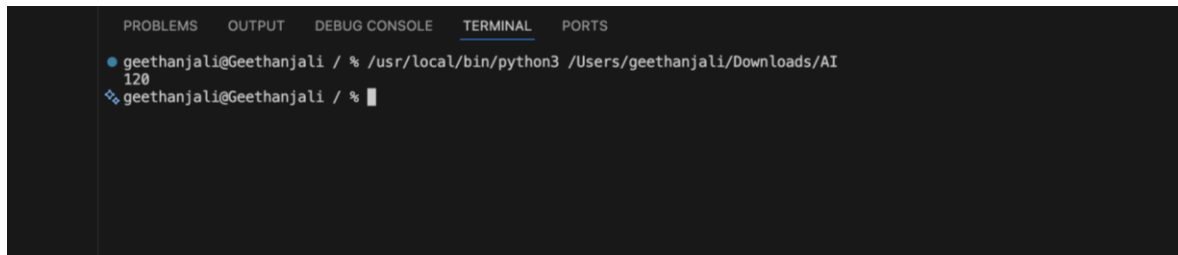
```
#Write a Python function to calculate the factorial of a given number.
Example:Input: 5
Output: 120

Add Context...
```

Copilot's suggestion:

```
> geethanjali > Downloads > AI > ...
1 def factorial(n):
2     """Calculate the factorial of a given number."""
3     if n < 0:
4         raise ValueError("Factorial is not defined for negative numbers")
5     if n == 0 or n == 1:
6         return 1
7     result = 1
8     for i in range(2, n + 1):
9         result *= i
10    return result
11
12
13 # Example usage
14 if __name__ == "__main__":
15     print(factorial(5)) # Output: 120
```

Output:

A terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, showing a prompt 'geethanjali@Geethanjali / %' followed by the command '/usr/local/bin/python3 /Users/geethanjali/Downloads/AI'. The output shows '120' and a second prompt 'geethanjali@Geethanjali / %'.

Compare the generated code with a zero-shot solution.

In the **zero-shot approach** the AI was only given a basic instruction to write a factorial program. Because no example was provided, the generated code worked only for normal positive numbers. It did not clearly handle negative numbers or special cases.

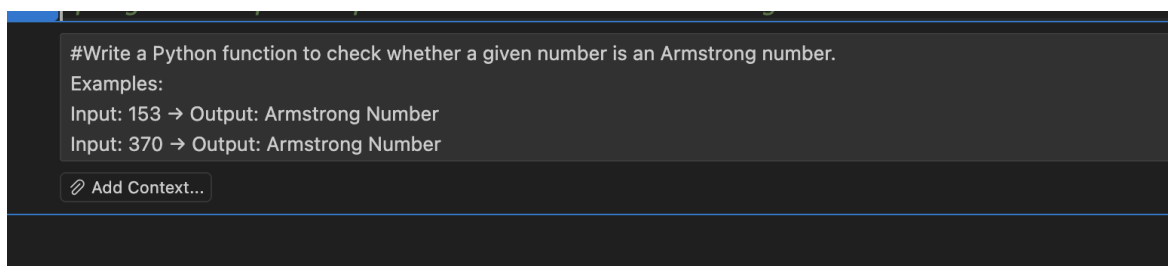
In the **one-shot approach** one input-output example was given (5 to 120). This helped the AI understand exactly what was expected. As a result, the generated code was clearer and more accurate. It also included a check for negative numbers, which improved correctness.

Examine improvements in clarity and correctness.

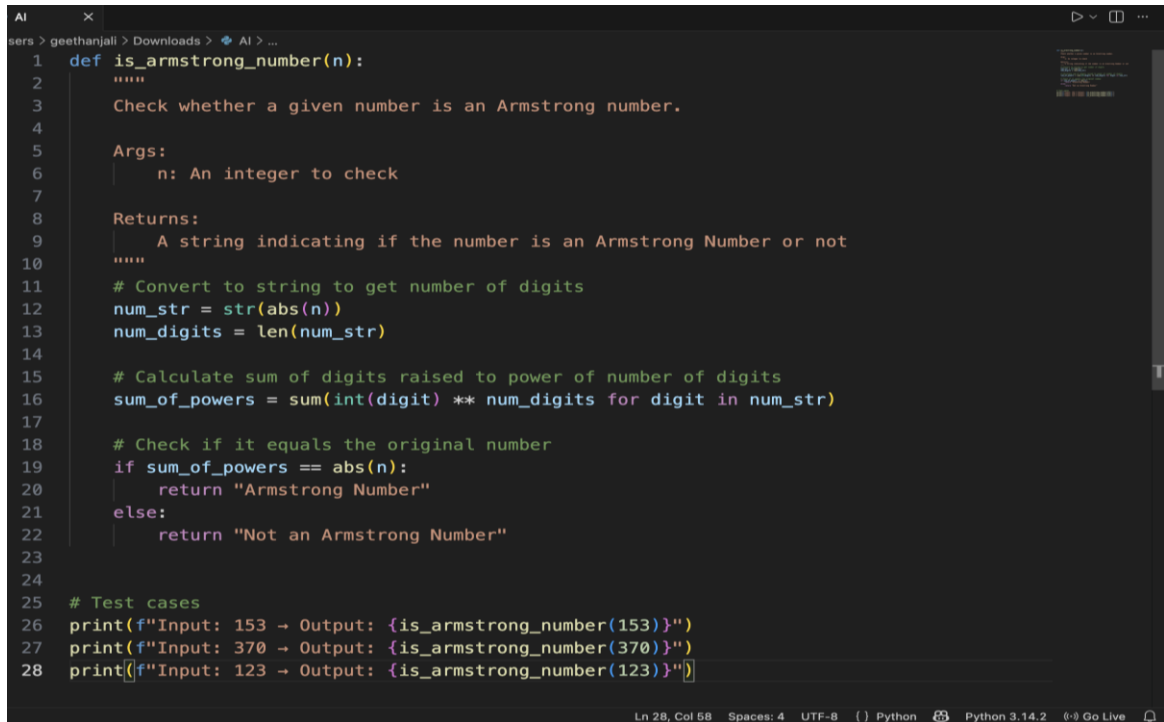
Overall, the **one-shot solution was better than the zero-shot solution**. Providing a single example helped the AI generate cleaner, more reliable, and easier-to-understand code.

Question 3: Few-Shot Prompting – Armstrong Number Check

Prompt:

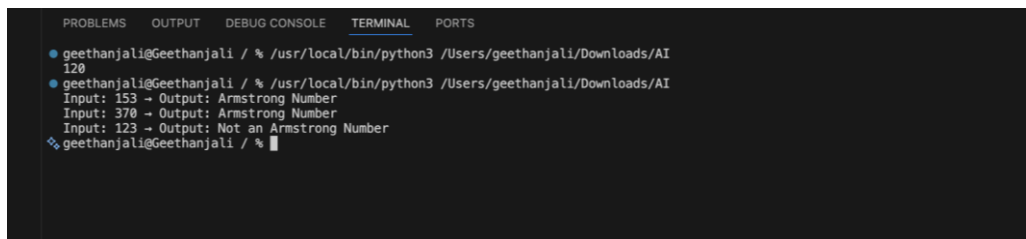
A dark-themed text area containing the following prompt: '#Write a Python function to check whether a given number is an Armstrong number. Examples: Input: 153 → Output: Armstrong Number Input: 370 → Output: Armstrong Number'. Below the text is a button labeled 'Add Context...'.

Copilot's suggestion:



```
1 def is_armstrong_number(n):
2     """
3     Check whether a given number is an Armstrong number.
4
5     Args:
6         n: An integer to check
7
8     Returns:
9         A string indicating if the number is an Armstrong Number or not
10    """
11    # Convert to string to get number of digits
12    num_str = str(abs(n))
13    num_digits = len(num_str)
14
15    # Calculate sum of digits raised to power of number of digits
16    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
17
18    # Check if it equals the original number
19    if sum_of_powers == abs(n):
20        return "Armstrong Number"
21    else:
22        return "Not an Armstrong Number"
23
24
25 # Test cases
26 print(f"Input: 153 → Output: {is_armstrong_number(153)}")
27 print(f"Input: 370 → Output: {is_armstrong_number(370)}")
28 print(f"Input: 123 → Output: {is_armstrong_number(123)}")
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
120
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Input: 153 → Output: Armstrong Number
Input: 370 → Output: Armstrong Number
Input: 123 → Output: Not an Armstrong Number
geethanjali@Geethanjali / %
```

Influence of Multiple Examples:

Providing multiple examples helped the AI clearly understand:

What an Armstrong number is

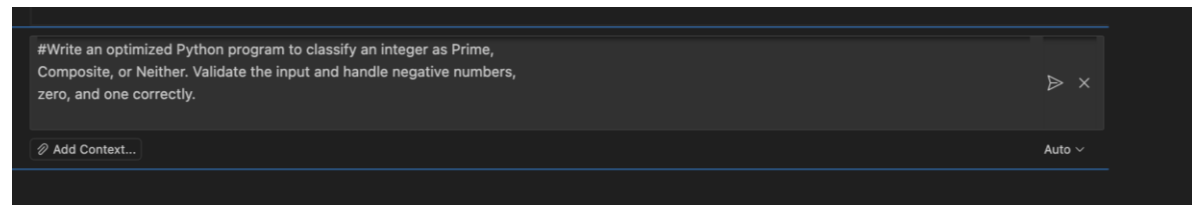
The expected output format

The logic required to compute digit powers

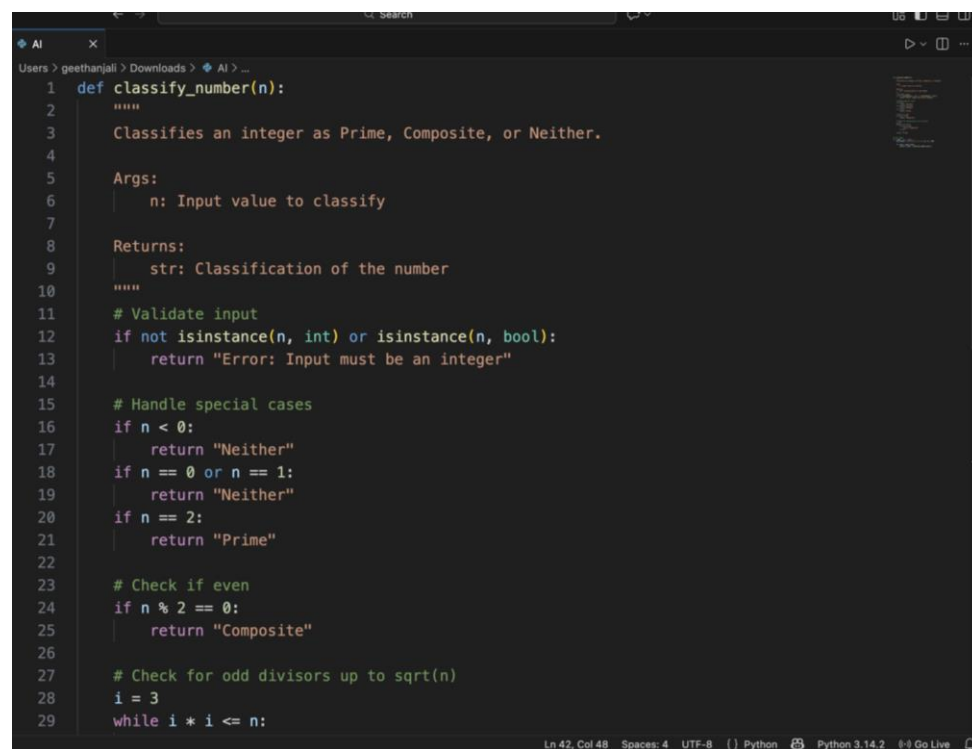
Generated code was **more structured, readable, and accurate** compared to zero shot and one-shot prompting.

Question 4: Context-Managed Prompting (Optimized Number Classification)

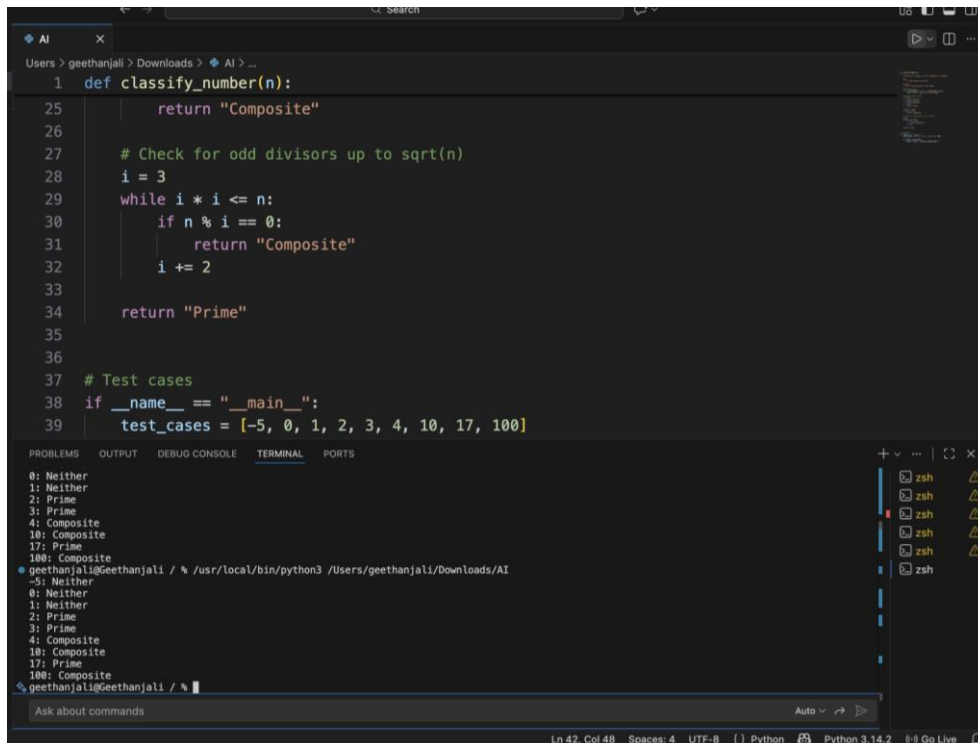
Prompt:



Copilot's suggestion:



Output:



```
1 def classify_number(n):
25     return "Composite"
26
27     # Check for odd divisors up to sqrt(n)
28     i = 3
29     while i * i <= n:
30         if n % i == 0:
31             return "Composite"
32         i += 2
33
34     return "Prime"
35
36
37 # Test cases
38 if __name__ == "__main__":
39     test_cases = [-5, 0, 1, 2, 3, 4, 10, 17, 100]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
0: Neither
1: Neither
2: Prime
3: Prime
4: Composite
10: Composite
17: Prime
100: Composite
geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
-5: Neither
0: Neither
1: Neither
2: Prime
3: Prime
4: Composite
10: Composite
17: Prime
100: Composite
% geethanjali@geethanjali / %
```

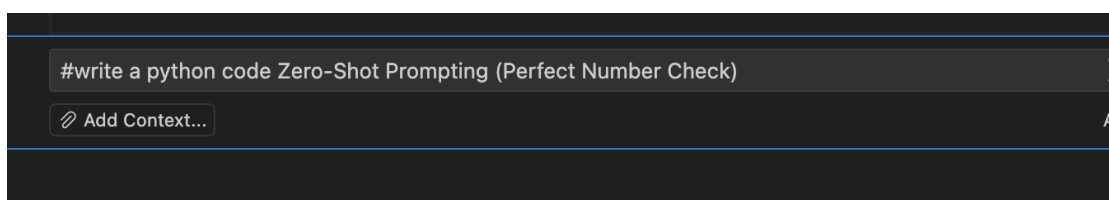
Ln 42, Col 48 Spaces: 4 UTF-8 Python Python 3.14.2

Explanation:

- The function first checks if the number is less than 2, returning neither.
- it then checks if the number is 2 (the only even prime).
- If the number is even and greater than 2, it returns composite.
- For odd numbers, it only checks for factors up to the square root of n, which reduces the number of iterations significantly.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Prompt:



Copilot's suggestion:

```
Users > geethanjali > Downloads > AI > ...  
1 def is_perfect_number(n):  
2     if n < 1:  
3         return False  
4     divisors_sum = sum(i for i in range(1, n) if n % i == 0)  
5     return divisors_sum == n  
6  
7 # Example usage  
8 number = 28  
9 if is_perfect_number(number):  
10     print(f"{number} is a perfect number.")  
11 else:  
12     print(f"{number} is not a perfect number.")
```

Output:

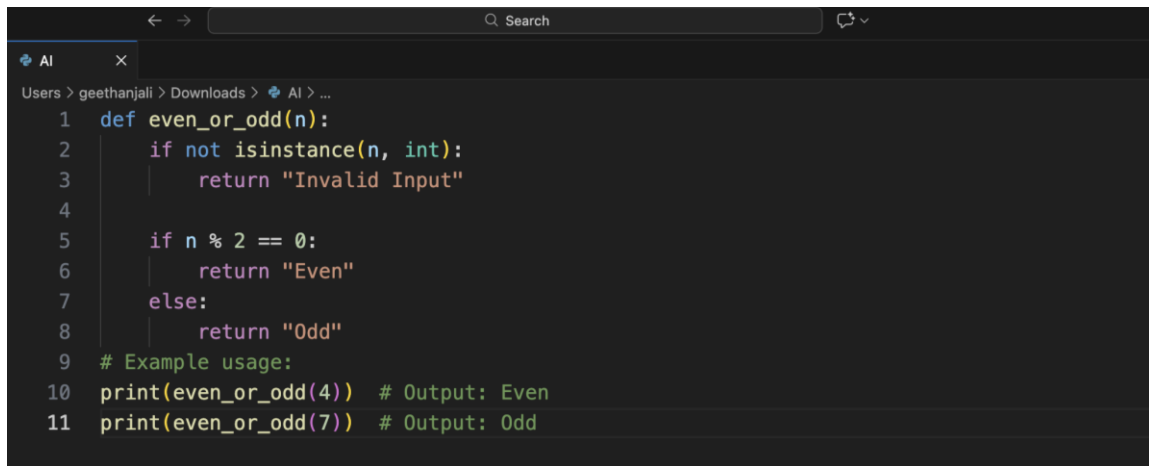
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI  
28 is a perfect number.  
❖ geethanjali@Geethanjali / %  
Ask about commands
```

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Prompt:

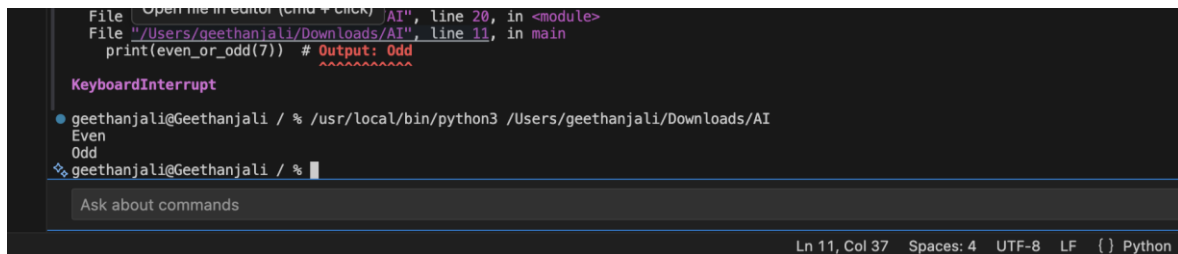
```
#Write a Python program to check whether a number is even or odd.  
Examples:  
Input: 8 → Output: Even  
Input: 15 → Output: Odd  
Add Context... Aut
```

Copilot's suggestion:



```
1 def even_or_odd(n):
2     if not isinstance(n, int):
3         return "Invalid Input"
4
5     if n % 2 == 0:
6         return "Even"
7     else:
8         return "Odd"
9
10 # Example usage:
11 print(even_or_odd(4)) # Output: Even
12 print(even_or_odd(7)) # Output: Odd
```

Output:



```
File "Open file in editor (cmd + click)" "AI", line 20, in <module>
File "/Users/geethanjali/Downloads/AI", line 11, in main
print(even_or_odd(7)) # Output: Odd
KeyboardInterrupt
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Even
Odd
geethanjali@Geethanjali / %
Ask about commands
Ln 11, Col 37 Spaces: 4 UTF-8 LF {} Python
```

Analysis: Effect of Few-Shot Prompting

- Examples clearly guided output format.
- Input validation was correctly added.
- Code is simple, readable, and accurate.
- Negative numbers are handled correctly.

Conclusion:

Among all prompting strategies:

- Zero-Shot - Basic but limited
- One-Shot - Better clarity
- Few-Shot - High accuracy and robustness
- Context-Managed - Best optimization and real-world readiness